

प्रोग्रामिंग भाषा (Programming Language) क्या है?

प्रोग्रामिंग भाषा एक औपचारिक भाषा (formal language) है जिसका उपयोग कंप्यूटर से संवाद (communication) करने और उसे निर्देश (instructions) देने के लिए किया जाता है। यह कंप्यूटर सॉफ्टवेयर और एप्लिकेशन विकसित करने के लिए आवश्यक होती है।

प्रोग्रामिंग भाषा कोडिंग सिंटैक्स (syntax) और नियमों का एक सेट प्रदान करती है, जिससे प्रोग्रामर (developer) कंप्यूटर को बताता है कि उसे क्या करना है।

प्रोग्रामिंग भाषाओं के प्रकार

प्रोग्रामिंग भाषाएँ मुख्य रूप से दो प्रकार की होती हैं:

1. लो-लेवल लैंग्वेज (Low-Level Language)

यह कंप्यूटर हार्डवेयर के अधिक नजदीक होती है और इसे मशीन कोड (binary code) के करीब माना जाता है। इसमें प्रोग्राम लिखना कठिन होता है, लेकिन यह तेज और प्रभावी होती है।

(a) मशीन लैंग्वेज (Machine Language)

- यह केवल 0 और 1 (बाइनरी कोड) में होती है।
- सीधे हार्डवेयर से संवाद करती है।
- समझना और लिखना कठिन होता है।

(b) असेम्बली लैंग्वेज (Assembly Language)

- इसमें संक्षिप्त कोड (mnemonics) का उपयोग होता है, जैसे कि MOV, ADD, SUB।
- यह मशीन लैंग्वेज की तुलना में पढ़ने और लिखने में आसान होती है।
- इसे असेम्बलर (Assembler) के माध्यम से मशीन कोड में बदला जाता है।

2. हाई-लेवल लैंग्वेज (High-Level Language)

- यह मनुष्यों के लिए पढ़ने और लिखने में आसान होती है।
- यह अंग्रेजी जैसे कीवर्ड (keywords) और सिंटैक्स का उपयोग करती है।
- इसे कंप्यूटर समझ सके, इसके लिए इसे कंपाइलर या इंटरप्रेटर द्वारा मशीन लैंग्वेज में बदला जाता है।

हाई-लेवल लैंग्वेज के उदाहरण:

- प्रक्रियात्मक (Procedural) – C, FORTRAN, COBOL
- ऑब्जेक्ट-ओरिएंटेड (Object-Oriented) – C++, Java, Python
- स्क्रिप्टिंग (Scripting) – JavaScript, PHP, Python
- फंक्शनल (Functional) – Lisp, Haskell

प्रोग्रामिंग की विशेषताएँ (Characteristics of Programming)

प्रोग्रामिंग एक प्रक्रिया है जिसमें कंप्यूटर को निर्देश (instructions) दिए जाते हैं ताकि वह कोई विशिष्ट कार्य कर सके। एक अच्छे प्रोग्राम की कुछ महत्वपूर्ण विशेषताएँ होती हैं, जो इसे प्रभावी और उपयोगी बनाती हैं।

1. स्पष्टता (Clarity)

प्रोग्राम का कोड सरल, सुव्यवस्थित और समझने में आसान होना चाहिए ताकि अन्य प्रोग्रामर इसे आसानी से पढ़ और संशोधित कर सकें।

2. दक्षता (Efficiency)

प्रोग्राम को कम समय (execution time) और कम मेमोरी (memory usage) में कार्य करना चाहिए ताकि सिस्टम के संसाधनों (resources) का अधिकतम उपयोग हो सके।

3. परीक्षणियता (Testability)

एक अच्छा प्रोग्राम आसानी से परीक्षण (testing) योग्य होता है, जिससे उसमें मौजूद बग (bugs) और त्रुटियों (errors) को आसानी से पहचाना और ठीक किया जा सके।

4. पुनः उपयोगिता (Reusability)

यदि कोड को बार-बार उपयोग किया जा सकता है (जैसे फ़ंक्शन या मॉड्यूल), तो यह कोडिंग समय बचाने में मदद करता है।

5. स्थिरता (Reliability)

प्रोग्राम को विभिन्न इनपुट और परिस्थितियों में स्थिरता से कार्य करना चाहिए और गलत इनपुट पर भी क्रैश (crash) नहीं होना चाहिए।

6. पोर्टेबिलिटी (Portability)

एक अच्छा प्रोग्राम विभिन्न प्लेटफॉर्म (Windows, Linux, Mac) पर आसानी से चलने योग्य होना चाहिए।

7. विस्तार क्षमता (Scalability)

प्रोग्राम को बिना अधिक बदलाव किए बड़े डेटा और अधिक उपयोगकर्ताओं के साथ काम करने में सक्षम होना चाहिए। ये सभी गुण एक प्रभावी और पेशेवर प्रोग्रामिंग सिस्टम के लिए आवश्यक होते हैं।

C भाषा का परिचय (Introduction to C)

- C भाषा 1972 में डेनिस रिची (Dennis Ritchie) द्वारा विकसित की गई थी।
- यह एक प्रक्रियात्मक (Procedural) प्रोग्रामिंग भाषा है।
- इसका उपयोग ऑपरेटिंग सिस्टम, डेटाबेस, कंपाइलर, एम्बेडेड सिस्टम आदि में किया जाता है।
- यह मिड-लेवल लैंग्वेज (Middle-Level Language) मानी जाती है क्योंकि इसमें लो-लेवल और हाई-लेवल दोनों विशेषताएँ होती हैं।
- यह बहुत तेज और प्रभावी होती है, इसलिए इसे "मदर ऑफ प्रोग्रामिंग लैंग्वेजेस" भी कहा जाता है।

C भाषा के मुख्य फीचर्स:

- तेज और शक्तिशाली
- पोर्टेबल (Portable)
- विस्तृत लाइब्रेरी सपोर्ट
- डायरेक्ट हार्डवेयर एक्सेस

C++ भाषा का परिचय (Introduction to C++)

- C++ को 1983 में ब्जार्ने स्ट्रॉस्ट्रुप (Bjarne Stroustrup) ने विकसित किया था।
- यह C का उन्नत (enhanced) संस्करण है, जिसमें ऑब्जेक्ट-ओरिएंटेड प्रोग्रामिंग (OOP) की सुविधा दी गई है।
- इसका उपयोग सॉफ्टवेयर डेवलपमेंट, गेम डेवलपमेंट, सिस्टम प्रोग्रामिंग आदि में किया जाता है।
- यह हाई-लेवल और लो-लेवल दोनों का मिश्रण है, जिससे इसे मल्टी-पैराडाइम लैंग्वेज कहा जाता है।

C++ भाषा के मुख्य फीचर्स:

- ऑब्जेक्ट-ओरिएंटेड – क्लास, ऑब्जेक्ट, इनकैप्सुलेशन, इनहेरिटेंस, पॉलीमॉर्फिज्म आदि।
- फास्ट और एफिशिएंट – लो-लेवल मेमोरी एक्सेस और डायरेक्ट हार्डवेयर कंट्रोल।
- मल्टी-पैराडाइम सपोर्ट – OOP, जनरिक प्रोग्रामिंग, फंक्शनल प्रोग्रामिंग।
- क्रॉस-प्लेटफॉर्म सपोर्ट – विभिन्न ऑपरेटिंग सिस्टम पर रन कर सकती है।

C और C++ के बीच अंतर

विशेषता	C	C++
प्रोग्रामिंग पैराडिग	प्रोसीजरल (Procedural)	ऑब्जेक्ट-ओरिएंटेड + प्रोसीजरल (OOP + Procedural)
मुख्य फोकस	फ़ंक्शन्स पर	ऑब्जेक्ट्स और क्लासेस पर
एन्कैप्सुलेशन	उपलब्ध नहीं	उपलब्ध (क्लासेस और ऑब्जेक्ट्स के माध्यम से)
इनहेरिटेंस (Inheritance)	नहीं	हाँ
ओवरलोडिंग (Overloading)	नहीं	हाँ (फ़ंक्शन और ऑपरेटर ओवरलोडिंग)
कोड की सुरक्षा	कम	ज्यादा (डेटा हाइडिंग की सुविधा)

यूज़ केस	सिस्टम प्रोग्रामिंग, एम्बेडेड सिस्टम	गेम डेवलपमेंट, GUI एप्लिकेशन, सिस्टम सॉफ्टवेयर
----------	--------------------------------------	--

KTU

एल्गोरिदम (Algorithm) क्या है?

एल्गोरिदम एक क्रमबद्ध (Step-by-step) प्रक्रिया होती है, जिसका उपयोग किसी समस्या को हल करने के लिए किया जाता है। इसे कंप्यूटर साइंस और गणित में विशेष रूप से इस्तेमाल किया जाता है।

सरल शब्दों में: "एल्गोरिदम वह विधि (Method) है जो हमें किसी कार्य को चरणबद्ध (Step-by-step) तरीके से करने की प्रक्रिया बताती है।"

एल्गोरिदम की विशेषताएँ (Characteristics of Algorithm)

इनपुट (Input): एल्गोरिदम में कुछ इनपुट दिए जाते हैं।

आउटपुट (Output): यह निश्चित रूप से एक परिणाम (Result) देता है।

स्पष्टता (Definiteness): प्रत्येक स्टेप स्पष्ट और सही तरीके से परिभाषित होता है।

सीमितता (Finiteness): एल्गोरिदम सीमित चरणों (Steps) में समाप्त हो जाता है।

प्रभावशीलता (Effectiveness): एल्गोरिदम को व्यावहारिक रूप से लागू किया जा सकता है।

एल्गोरिदम का एक सरल उदाहरण

समस्या: दो संख्याओं का योग निकालना

एल्गोरिदम:

चरण 1: शुरू करें (Start)

चरण 2: दो संख्याएँ इनपुट करें (A और B)

चरण 3: उनका योग निकालें ($SUM = A + B$)

चरण 4: परिणाम प्रदर्शित करें (Print SUM)

चरण 5: समाप्त करें (End)

Step 1: Start

Step 2: Input two numbers (A and B)

Step 3: Calculate their sum ($SUM = A + B$)

Step 4: Display the result (Print SUM)

Step 5: End

2. Algorithm to Find the Largest of Two Numbers

Problem: Find the larger number between two given numbers.

Algorithm:

1. Start
2. Input two numbers (A and B)
3. If $A > B$, print "A is larger"
4. Else, print "B is larger"
5. End

3. Algorithm to Check Whether a Number is Even or Odd

Problem: Determine whether a given number is even or odd.

Algorithm:

1. Start
2. Input a number (N)
3. If $N \% 2 == 0$, print "Even"
4. Else, print "Odd"
5. End

फ्लोचार्ट (Flowchart) क्या है?

फ्लोचार्ट एक एल्गोरिदम का चित्रात्मक (Graphical) प्रतिनिधित्व है। इसमें विभिन्न चरणों (Steps) को अलग-अलग आकृतियों (Shapes) द्वारा दर्शाया जाता है। इन आकृतियों को तीर (Arrows) के माध्यम से जोड़ा जाता है, जिससे डेटा के प्रवाह (Flow of Data) को दिखाया जाता है।

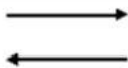
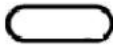
फ्लोचार्ट का महत्व

फ्लोचार्ट किसी प्रोग्राम या प्रक्रिया में डेटा के प्रवाह को समझने में मदद करता है। इसका उपयोग जटिल एल्गोरिदम (Complex Algorithms) को सरल रूप में प्रस्तुत करने और प्रोग्राम की डिबगिंग (Debugging) के लिए किया जाता है।

यह न केवल प्रोग्रामर्स बल्कि गैर-तकनीकी (Non-Technical) लोगों को भी समाधान की संरचना (Structure) और तर्क (Logic) को आसानी से समझने में मदद करता है।

फ्लोचार्ट के प्रतीक (Symbols of Flowchart)

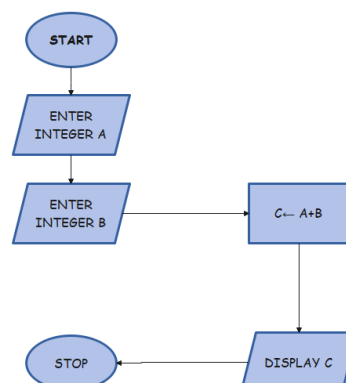
फ्लोचार्ट में विभिन्न प्रतीकों (Symbols) का उपयोग किया जाता है, जो विभिन्न चरणों (Steps) को दर्शाते हैं। ये प्रतीक तीरों (Arrows) से जुड़े होते हैं, जो डेटा के प्रवाह को दिखाते हैं। नीचे प्रमुख फ्लोचार्ट प्रतीकों का विवरण दिया गया है—

Symbol	Symbol Name	Description
	Flow Lines	Used to connect symbols
	Terminal	Used to start, pause or halt in the program logic
	Input/output	Represents the information entering or leaving the system
	Processing	Represents arithmetic and logical instructions
	Decision	Represents a decision to be made
	Connector	Used to Join different flow lines
	Sub function	used to call function

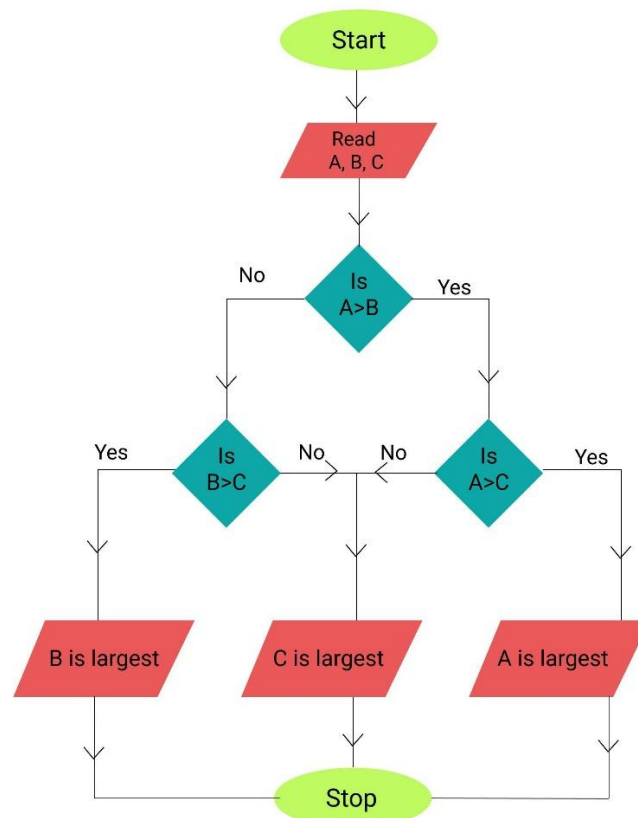
Examples of flow chart

1.

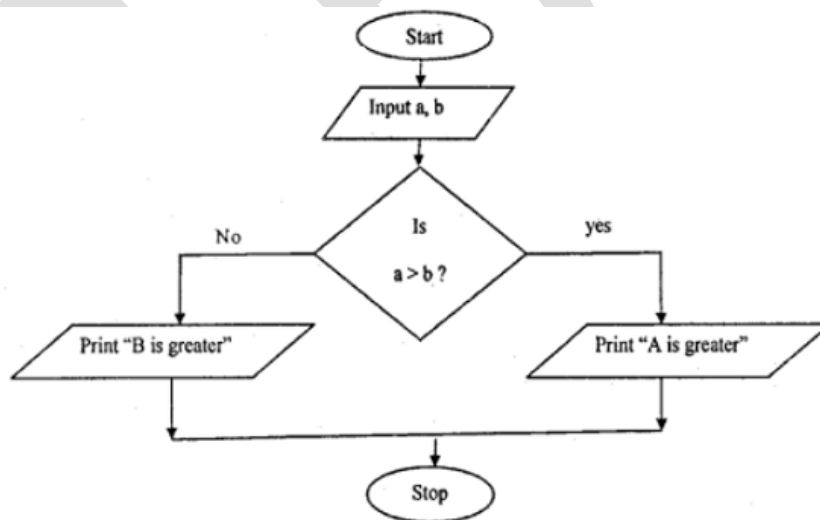
Find the Sum of Two Numbers Entered



2. Draw a flowchart to find the greatest of the three numbers



3. Write a flowchart to find the greater of two numbers?



Variable

C++ में **वेरिएबल (Variable)** एक ऐसी संरचना है जिसका उपयोग डेटा को स्टोर करने के लिए किया जाता है। जब हम प्रोग्राम लिखते हैं, तो हमें डेटा को संग्रहीत करने के लिए वेरिएबल की आवश्यकता होती है। वेरिएबल एक नाम (identifier) होता है, जिसके जरिए हम डेटा तक पहुँच सकते हैं।

वेरिएबल की परिभाषा: वेरिएबल एक मेमोरी लोकेशन है जिसमें किसी विशेष प्रकार का डेटा संग्रहीत किया जाता है। प्रत्येक वेरिएबल का एक प्रकार (data type) होता है जो यह निर्धारित करता है कि वेरिएबल में कौन सा प्रकार का डेटा स्टोर किया जा सकता है (जैसे, पूर्णांक, दशमलव, आदि)।

C++ में वेरिएबल के प्रकार (Data Types in C++):

C++ में वेरिएबल के कई प्रकार होते हैं, जिनका उपयोग विभिन्न प्रकार के डेटा को स्टोर करने के लिए किया जाता है। इन्हें दो श्रेणियों में बाँटा जा सकता है:

1. प्रिमिटिव डेटा प्रकार (Primitive Data Types):

- **int:** यह पूर्णांक (integer) मान को स्टोर करने के लिए उपयोग किया जाता है। जैसे, 5, -3, 100 आदि।
- **float:** यह दशमलव संख्या (floating-point numbers) को स्टोर करने के लिए उपयोग किया जाता है। जैसे, 5.23, -3.14 आदि।
- **double:** यह भी दशमलव संख्या को स्टोर करता है, लेकिन इसकी precision (सटीकता) float से अधिक होती है।
- **char:** यह एक सिंगल कैरेक्टर (character) को स्टोर करने के लिए उपयोग किया जाता है। जैसे, 'a', 'A', '1' आदि।
- **bool:** यह केवल दो मानों को स्टोर करता है, true (सच) और false (झूठ)।
- **wchar_t:** यह एक वाइड कैरेक्टर (विशाल अक्षर) को स्टोर करता है, जैसे यूनिकोड मान।

2. कोणिक डेटा प्रकार (Derived Data Types):

- **Array:** यह एक ही प्रकार के डेटा के संग्रह को स्टोर करता है। उदाहरण के लिए, int array[5]।
- **Pointer:** यह किसी अन्य वेरिएबल का पता (address) स्टोर करता है।
- **Reference:** यह किसी अन्य वेरिएबल का एक संदर्भ (reference) होता है।

वेरिएबल का निर्धारण (Declaration of Variables):

C++ में वेरिएबल को घोषित करने का तरीका निम्नलिखित होता है:

Syntax:

data_type variable_name;

example/उदाहरण:

int x; // x नाम का एक integer वेरिएबल घोषित किया गया है

float y; // y नाम का एक float वेरिएबल घोषित किया गया है

char c; // c नाम का एक character वेरिएबल घोषित किया गया है

वेरिएबल का इनिशियलाइजेशन (Initialization of Variables):

वेरिएबल को घोषित करने के बाद, उसे एक मान (value) सौंपना होता है, जिसे इनिशियलाइजेशन कहा जाता है।

int x = 10; // x को 10 के साथ इनिशियलाइज़ किया गया है

float y = 3.14; // y को 3.14 के साथ इनिशियलाइज़ किया गया है

वेरिएबल के नियम:

1. **वेरिएबल का नाम अंक से शुरू नहीं हो सकता:** वेरिएबल का नाम हमेशा अक्षर (alphabet) से या अंडरस्कोर (underscore) से शुरू होना चाहिए, न कि किसी संख्या से। उदाहरण: 1age गलत है, जबकि age1 सही है।
2. **वेरिएबल के नाम में खाली स्थान (space) नहीं होना चाहिए:** वेरिएबल का नाम बिना किसी खाली स्थान के होना चाहिए। उदाहरण: my variable गलत है, जबकि my_variable सही है।
3. **वेरिएबल के नाम में केवल अक्षर, अंक और अंडरस्कोर (underscore) हो सकते हैं:** वेरिएबल का नाम केवल अक्षर (a-z, A-Z), अंक (0-9), और अंडरस्कोर (underscore) से बना हो सकता है। उदाहरण: my-variable गलत है, जबकि my_variable सही है।

- C++ में वेरिएबल नाम केस-सेंसिटिव होते हैं: इसका मतलब है कि age और Age अलग-अलग वेरिएबल माने जाएंगे। उदाहरण: int age और int Age दोनों अलग-अलग वेरिएबल होंगे।
- कीवर्ड्स को वेरिएबल नाम के रूप में उपयोग नहीं किया जा सकता: C++ में कीवर्ड्स जैसे int, float, class, if, public, आदि को वेरिएबल नाम के रूप में उपयोग नहीं किया जा सकता। उदाहरण: int int = 10; या float class = 5.5; गलत है, क्योंकि int और class C++ के कीवर्ड्स हैं

वेरिएबल के नियम:

- वेरिएबल का नाम अंक से शुरू नहीं हो सकता।
- वेरिएबल के नाम में खाली स्थान (space) नहीं होना चाहिए।
- वेरिएबल के नाम में केवल अक्षर, अंक और अंडरस्कोर () हो सकते हैं।
- C++ में वेरिएबल नाम केस-सेंसिटिव होते हैं, यानी age और Age अलग-अलग वेरिएबल माने जाएंगे।
- कीवर्ड्स को वेरिएबल नाम के रूप में उपयोग नहीं किया जा सकता

C++ ऑपरेटर्स

C++ में ऑपरेटर्स विशेष प्रतीक (symbols) होते हैं जिनका उपयोग वेरिएबल्स और मानों (values) पर विभिन्न गणनाएँ (operations) करने के लिए किया जाता है। ऑपरेटर्स किसी भी प्रोग्रामिंग भाषा का एक महत्वपूर्ण हिस्सा होते हैं। C++ में कई प्रकार के ऑपरेटर्स होते हैं, जैसे कि अंकगणितीय (arithmetic), असाइनमेंट (assignment), बिटवाइज (bitwise), लॉजिकल (logical), और अन्य।

C++ ऑपरेटर्स के प्रकार:

C++ में निम्नलिखित प्रकार के ऑपरेटर्स होते हैं:

- अंकगणितीय (Arithmetic) ऑपरेटर्स
- तुलनात्मक (Relational) ऑपरेटर्स
- लॉजिकल (Logical) ऑपरेटर्स
- बिटवाइज (Bitwise) ऑपरेटर्स
- असाइनमेंट (Assignment) ऑपरेटर्स
- टर्नरी (Ternary) ऑपरेटर्स
- अन्य (Miscellaneous) ऑपरेटर्स

1. अंकगणितीय (Arithmetic) ऑपरेटर्स

ये ऑपरेटर्स गणितीय गणनाएँ करने के लिए उपयोग किए जाते हैं। इन्हें दो भागों में बाँटा जाता है: बाइनरी और यूनरी ऑपरेटर्स।

ऑपरेटर	नाम	विवरण
+	जोड़ (Addition)	दो संख्याओं को जोड़ता है
-	घटाव (Subtraction)	एक संख्या से दूसरी संख्या घटाता है
*	गुणा (Multiplication)	दो संख्याओं का गुणा करता है
/	भाग (Division)	पहली संख्या को दूसरी संख्या से विभाजित करता है
%	मॉड्यूलस (Modulus)	भागफल का शेष भाग निकालता है
++	वृद्धि (Increment)	मान में 1 जोड़ता है
--	कमी (Decrement)	मान में 1 घटाता है

उदाहरण:

```
#include <iostream>
using namespace std;
int main() {
    int x = 15, y = 4;
    cout << "Addition: " << (x + y) << endl;
    cout << "Subtraction: " << (x - y) << endl;
    cout << "Multiplication: " << (x * y) << endl;
    cout << "Division: " << (x / y) << endl;
    cout << "Modulus: " << (x % y) << endl;
    cout << "Increment: " << (++x) << endl;
    cout << "Decrement: " << (--x) << endl;
    return 0;
}
```

2. तुलनात्मक (Relational) ऑपरेटर्स

ये ऑपरेटर्स दो मानों की तुलना करने के लिए उपयोग किए जाते हैं और true (1) या false (0) रिटर्न करते हैं।

ऑपरेटर	नाम	विवरण
==	बराबर (Equal To)	दो मान समान हैं या नहीं
!=	असमान (Not Equal To)	दो मान अलग हैं या नहीं
>	बड़ा (Greater Than)	पहला मान दूसरे से बड़ा है या नहीं
<	छोटा (Less Than)	पहला मान दूसरे से छोटा है या नहीं
>=	बड़ा या बराबर (Greater Than or Equal To)	पहला मान दूसरे से बड़ा या बराबर है या नहीं
<=	छोटा या बराबर (Less Than or Equal To)	पहला मान दूसरे से छोटा या बराबर है या नहीं

उदाहरण:

```
#include <iostream>
using namespace std;
int main() {
    int a = 16, b = 7;
    cout << (a == b) << endl;
    cout << (a > b) << endl;
    cout << (a < b) << endl;
    return 0;
}
```

3. लॉजिकल (Logical) ऑपरेटर्स

ये ऑपरेटर्स लॉजिकल ऑपरेशन्स के लिए उपयोग किए जाते हैं।

ऑपरेटर	नाम	विवरण
&&	AND	जब दोनों शर्तें सत्य हों तब true देता है
	OR	यह तब true (1) देता है जब कोई एक या दोनों शर्तें सत्य (true) हों। अगर दोनों शर्तें असत्य (false) हैं, तो यह false (0) देता है।
!	NOT	दिए गए मान का विपरीत मान देता है

उदाहरण:

```
#include <iostream>
using namespace std;
int main() {
    int x = 18, y = 12;
    cout << (x && y) << endl;
    cout << (x || y) << endl;
    cout << (!y) << endl;
    return 0;
}
```

}

4. बिटवाइज (Bitwise) ऑपरेटर्स

ये ऑपरेटर्स बिट स्तर (bit level) पर कार्य करते हैं।

ऑपरेटर	नाम	विवरण
&	AND	दोनों बिट्स 1 हों तो ही 1 देता है
	OR	दोनों में से कोई भी 1 बिट्स 1 हो तो 1 देता है
^	XOR	भिन्न बिट्स पर 1 देता है
~	NOT	सभी बिट्स उलट देता है
<<	Left Shift	बाईं ओर बिट्स को शिफ्ट करता है
>>	Right Shift	दाईं ओर बिट्स को शिफ्ट करता है

5. असाइनमेंट (Assignment) ऑपरेटर्स

ये ऑपरेटर्स वेरिएबल में मान असाइन करने के लिए उपयोग किए जाते हैं।

ऑपरेटर	नाम	विवरण
=	असाइनमेंट	मान को असाइन करता है
+=	जोड़कर असाइन	$x = x + y$ के बराबर
-=	घटाकर असाइन	$x = x - y$ के बराबर
*=	गुणा करके असाइन	$x = x * y$ के बराबर
/=	भाग देकर असाइन	$x = x / y$ के बराबर

C++ Header Files (संक्षेप में):

C++ **हेडर फ़ाइलें** उन फ़ाइलों होती हैं जिनमें फ़ंक्शन्स, क्लासेस, और अन्य डेफिनिशन्स की घोषणाएँ होती हैं। इन्हें `#include` के जरिए प्रोग्राम में जोड़ा जाता है। हेडर फ़ाइलों का उपयोग कोड को मॉड्यूलर बनाने, पुनः उपयोग (reuse) सक्षम करने, और इंटरफेस और कार्यान्वयन (implementation) को अलग करने के लिए किया जाता है।

- **Standard Header Files:** जैसे `<iostream>`, `<cmath>`, `<string>`, आदि।
- **User-defined Header Files:** जैसे `"myheader.h"` जो उपयोगकर्ता द्वारा बनाई जाती हैं।

C++ Keywords (संक्षेप में):

C++ में **कीवर्ड्स** (Keywords) वे आरक्षित शब्द होते हैं जिनका भाषा में विशेष अर्थ होता है। ये शब्द C++ के विभिन्न घटकों (components) को परिभाषित करने के लिए उपयोग किए जाते हैं, जैसे `void`, `int`, `public`, आदि। इन शब्दों का उपयोग किसी वेरिएबल नाम, फ़ंक्शन नाम, या अन्य पहचानकर्ता (identifiers) के रूप में नहीं किया जा सकता।

अर्थात्, ये शब्द C++ भाषा के निर्माण और संरचना का हिस्सा होते हैं, और इन्हें सामान्य रूप से किसी अन्य उद्देश्य के लिए उपयोग नहीं किया जा सकता।

C++ **कीवर्ड्स** वे विशेष शब्द होते हैं जिनका C++ प्रोग्राम में विशेष अर्थ होता है। इन्हें किसी अन्य उद्देश्य के लिए उपयोग नहीं किया जा सकता। कुछ प्रमुख कीवर्ड्स:

1. **int:** पूर्णांक डेटा प्रकार।
2. **float:** दशमलव संख्या डेटा प्रकार।
3. **if, else:** शर्तों को नियंत्रित करने के लिए।
4. **while, for:** लूप्स के लिए।
5. **class:** कस्टम डेटा प्रकार बनाने के लिए।
6. **public, private:** क्लास में डेटा एक्सेस को नियंत्रित करने के लिए।
7. **return:** फ़ंक्शन से मान वापस भेजने के लिए।
8. **void:** जब फ़ंक्शन कोई मान वापस नहीं करता।
9. **new, delete:** डायनेमिक मेमोरी आवंटित और डिलीट करने के लिए।